

@prism project

Christophe BAL

14 Mar. 2026 – Version 2.1.0

The @prism project¹ provides 350 color palettes with native support for [CSS](#), [Lua](#), and [L^AT_EX](#), to help you create expressive content

Last changes

🔧 Fix.

- Online demo: an auto-update script for the `HTML`, `CSS` and `JavaScript` codes was missing.

🔨 Break.

- Palette purification.
 - Deprecated sublist exclusion due to the high overhead of manual reconstruction: return of the palettes `BlueDarkOrange12`, `BlueOrange8`, and `BlueOrange10`.
 - No more visual identical palettes (semi-automated process): `Cubehelix` ([Matplotlib](#)) was removed, as it is visually identical to `Classic` ([Cubehelix](#)).

🔄 Update.

- Manual.
 - The `Broc`, `GistRainbow`, and `Tofino` palettes have been removed from the cyclic category.
 - The `BlueDarkOrange12`, `BlueOrange8`, and `BlueOrange10` palettes have been added to the sequential category.
 - Added parameters allowing for the reconstruction of palettes omitted due to being shifted variants of existing ones.
 - Added informative appendix describing subpalettes.

¹The name comes from “*@·esthetic P·roducts for R·epresenting I·nformative S·cientific M·aps*”. This name is a double play on words: [1] a prism splits light into an informative spectrum, symbolizing how data are decomposed into meaningful color, and [2] “@” read as “at” indicates where the light meets the prism to be broken down into an informative spectrum.

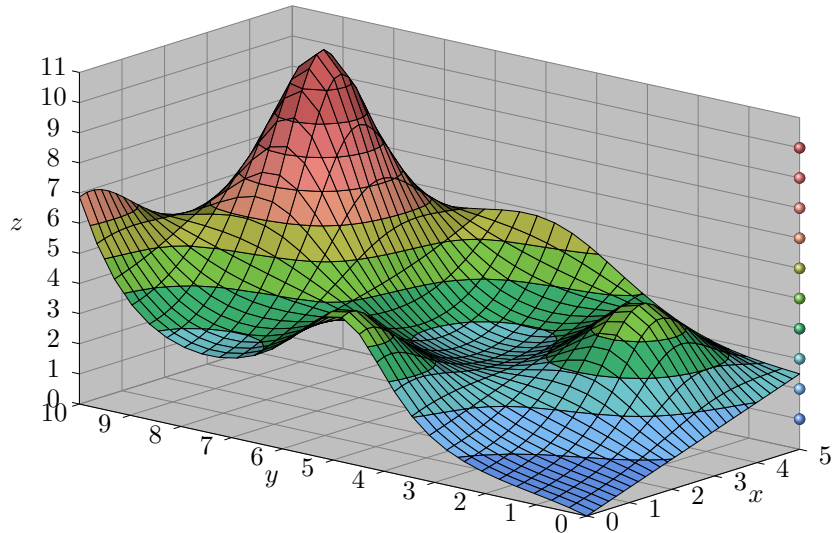
Contents

I. Motivations	3
II. Where do the color palettes come from?	4
1. Resources used	4
2. Palette integration	4
III. Reuse from...	5
IV. How to choose a palette?	5
V. Supported implementations	6
1. JSON, the versatile default format	6
2. CSS	6
3. LaTeX	7
a. Basic use	7
b. Creating palettes from scratch	7
c. Creating palettes from existing ones	8
d. Retrieving the internal definition of a palette	8
4. Lua	9
a. Basic use	9
b. Creating palettes from existing ones	9
VI. Contribute via Git	9
1. Complete the translations	9
a. The en folder	9
b. The changes folder	10
c. The status folder	10
d. The README.md and LICENCE.txt files	10
e. New translations	10
2. Improving the source code	10
VII. History	11
All palette names	14
Renamed palettes	17
Ignored palettes	19
• Excluded by design	19
• Unkept palettes	19
• Reconstructing unkept shifted palettes	19
Subpalettes	20
Palettes by category	21
• Colorblind palettes	21
• Cyclic palettes	23
• Dark palettes	24
• Divergent palettes	25
• Qualitative palettes	28
• Semantic palettes	32
• Sequential palettes	33
• Stepped palettes	40

I. Motivations

This project was born out of a desire to enhance `luadraw` with a set of color palettes to easily produce graphics like the 3D plot shown below. Despite the availability of palette generation tools such as `Coolors`, the choice has been made to offer pre-built palettes, because this serves several key purposes for non-designers.

- **Reduced cognitive load:** pre-configuration eliminates design decisions, allowing to focus on content.
- **Consistency:** it becomes easy to ensure visual coherence across documents and projects, regardless of technology.
- **Accessibility compliance:** some pre-configured palettes guarantee colorblind-safe combinations.
- **Perceptual uniformity:** most palettes ensure distinguishable and meaningful colors for data visualization, while others serve artistic purposes.



Technically, to generate the graphics in this section, `luadraw` is provided with a finite list of colors and uses linear interpolations to calculate the intermediate color values. In the example above, the finite color palette used is defined as follows.

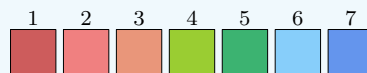


Using this palette, `luadraw` can produce the following spectrum, allowing us to create the graphic above.

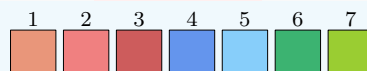


Note.

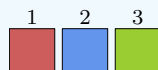
Using the `Lua` implementation of `@prism`, see the section `V-4`, via `luadraw`, we can create the palettes below made from the previous one named `'GeoRainbow'`. Each instruction used is given below each palette.



```
palCreateFromPal('GeoRainbow', {reverse = true})
```



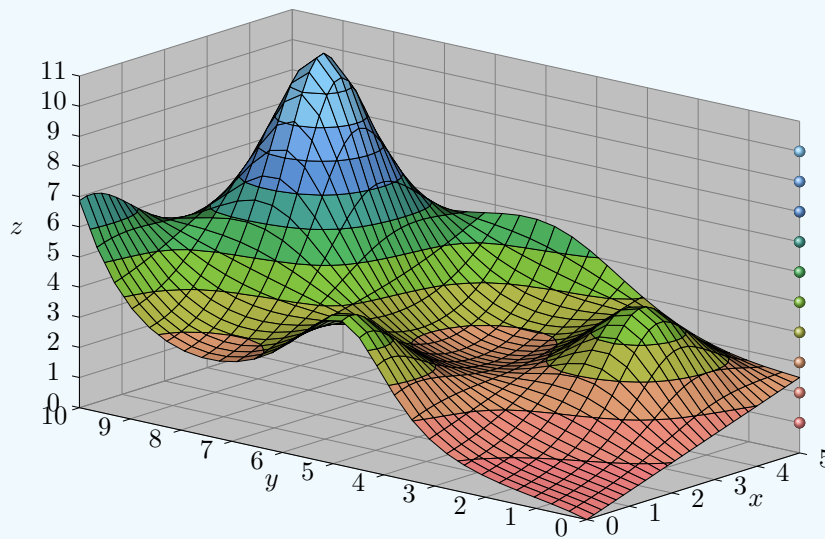
```
palCreateFromPal('GeoRainbow', {shift = 3})
```



```
palCreateFromPal('GeoRainbow', {extract = {7, 1, 4}})
```

This features provide remarkable creative flexibility: with the same surface as before, using the setting `palCreateFromPal('GeoRainbow', {extract = {6, 5, 4, 3, 1, 2}})` instead of

`palCreateFromPal('GeoRainbow')`, we change the visual tone, shifting from a seaside feel to a snow-covered world.



II. Where do the color palettes come from?

1. Resources used

In addition to its 10 creations (representing 3%), `@prism` incorporates 340 palettes from the following projects.

- `Asymptote` is used, but currently offers nothing beyond `Matplotlib` (despite different implementations).
- `CarbonPlan` provides palettes designed for data visualization..
- `CARTOColors` palettes are extracted from `Palettable` project.
- `CMasher` provides palettes designed for data visualization..
- `cmocean` palettes are extracted from `Palettable` project.
- `Colorbrewer` provides professional color palettes for mapping and data visualization.
- `Light and Bartlein` palettes are extracted from `Palettable` project.
- `Matplotlib` compiles color maps from diverse projects.
- `MyCarta` palettes are extracted from `Palettable` project.
- `Plotly` palettes are extracted from `Palettable` project.
- `Scientific Coulour Maps` provides palettes designed for colorblind accessibility.
- `Tableau` palettes are extracted from `Palettable` project.
- `Wes Anderson Palettes` palettes are extracted from `Palettable` project.

Note.

The following versions were used to build the current `@prism` release.

<code>Asymptote</code> : 2026-04-05	<code>Colorbrewer</code> : 2021-05-03
<code>CarbonPlan</code> : 2024-01-10	<code>Matplotlib</code> : 2026-04-08
<code>CARTOColors</code> : 2024-09-25	<code>Palettable</code> : 2025-08-23
<code>CMasher</code> : 2026-04-06	<code>Scientific Coulour Maps</code> : 2023-10-05

2. Palette integration

We retain only palettes that comply with the following rules.

- **No duplicates.** `@prism` maintains a one-to-one mapping² between names and palettes.³
- **No basic derived versions.** `@prism` omits palettes that can be reconstructed by shifting⁴ and/or reversing⁵ existing ones, because most `@prism` implementations offer an API to handle these transformations dynamically.
- **Reasonable palette sizes.** Palettes exceeding 512 colors are ignored⁶. This prevents compatibility issues with certain technologies, such as L^AT_EX, and provides a level of precision far beyond the needs of us mere mortals — or at least, those of us who aren't astronomers.

Note.

Adding new palettes to `@prism` is straightforward (no coding skills required). See section VI-2 to get started with any of the available technologies.

III. Reuse from...

Here are the key points to remember to use palettes offered by projects listed in the section II.

1. Shifted palettes⁷ and reversed ones⁸ are not included in `@prism`. In rare cases, some palettes are excluded because they appear redundant or are too large. All excluded palettes are listed in Appendix 3 (page 19).
2. `@prism` uses 'semantic' CamelCase naming. Therefore, palette names such as `berlin`, `gist_heat` and `Pinkgrey` become `Berlin`, `GistHeat` and `PinkGrey`. To avoid naming collisions, certain palettes have been renamed : a full list of these changes is provided in Appendix 2 (page 17). You can see the full list of names in Appendix 1 (page 14).

Caution.

Most `@prism` implementations add the `pal` prefix to standardized CamelCase names. See the section V.

Note.

Most `@prism` implementations provide methods to easily obtain reversed palettes, sub-palettes, and color-shifted palettes. See the section V.

IV. How to choose a palette?

Here are a few tips to help you select your color palette.

1. An [online demo of `@prism`](#)⁹ lets you intuitively browse palettes with the following features.
 - **Random discovery:** a 'random' button to explore new palettes.
 - **Category filtering:** quickly narrow down choices by category.
 - **Palette export:** download specific palette files directly.
2. Appendix 5 (page 21) details all palettes by category, showing discrete definitions (semantic/qualitative) and continuous spectra.

²Some `Matplotlib` palettes are duplicated, likely for historical reasons.

³This does not exclude visually similar palettes with distinct names, which are preserved to honor specific design intents. However, if you find palettes with indistinguishable color sets, at least to the naked eye, please report this as a bug.

⁴`Colorcet` offers shifted palettes whose names use an integer suffix.

⁵Most `Matplotlib` colormaps include a reversed version suffixed with `_r`, possibly for performance reasons.

⁶`CMasher` offers the `neutral` palette, which contains 2560 colors

⁷By design, `Colorcet` proposes several shifted palettes.

⁸`Matplotlib` palettes with names ending in `_r` are reversed versions.

⁹<https://projctmbc.github.io/for-writing/>

V. Supported implementations

All implementations are located in the `products` folder. Each implementation provides the following features.

- Modular palette formats (one file per palette).
- Palette definitions in both high-fidelity (original size) and small (currently 40 colors at most) sizes.

Most implementations also feature the API explained below.

- Select specific colors from an existing palette using their indices.
- Shift the palette left (negative value) or right (positive value) by any number of steps.
- Reverse the order of the colors.

Important.

To explain how new palettes can be built, we will refer to the colors in the standard palette as `coul_1`, `coul_2`, etc., and suppose that the extracted indices are `{1, 3, 6, 9}`, the shift used is `1+`, and the `reverse` option is enabled. The new palette will then be built sequentially as follows: first `{coul_1, coul_3, coul_6, coul_9}` (extraction), second `{coul_9, coul_1, coul_3, coul_6}` (shift to the right), and finally `{coul_6, coul_3, coul_1, coul_9}` (reverse).

Note.

Extra features are limited to discrete palette operations. For example, color interpolation is not provided, as this is usually handled out of the box by visualization and formatting tools.

1. JSON, the versatile default format

The JSON product enables seamless `@prism` palette integration for unsupported programming languages. As shown below, palettes are defined using nested arrays of three-component floats.

```
[
  [0.4980392157, 0.7882352941, 0.4980392157],
  [0.7450980392, 0.6823529412, 0.831372549],
  ...
]
```

2. CSS

Note.

The `HTML`, `CSS`, and `JavaScript` files for product development and demonstration were created using `Claude` and `Gemini AI` assistants.

Each palette color is defined as an individual variable named according to the pattern `--pal<name>-<nb>`, where `<name>` is the standard palette name and `<nb>` is the desired index ranging. Each palette color variable is defined as an RGB value using percentage notation. For instance, the following snippet is an excerpt from `palettes-hf/Accent.css`.

```
:root {
  --palAccent-1: rgb(49.803922% 78.823529% 49.803922%);
  --palAccent-2: rgb(74.509804% 68.235294% 83.137255%);
  ...
}
```

The following example illustrates how to generate gradient variables via selective color extraction and custom reordering, while using a standalone color for warning text.

```
:root {
  --transformed-accent-gradient: linear-gradient(
    90deg,
    var(--palAccent-6),
```

```

    var(--palAccent-3),
    var(--palAccent-1)
);
}

.transformed-accent-gradient {
    background: var(--transformed-accent-gradient);
}

.warning-text {
    color: var(--palAccent-5);
}

```

3. LaTeX

a. Basic use

Accessing a single palette color is straightforward: use `\palUse{<name>}{<index>}` where `<name>` is the standard palette name (without prefix), and `<index>` is the color number. For example, `\palUse{Accent}{8}` is the eighth color of the `Accent` palette, an `xcolor` format color that can be easily used as shown in the following compilable example.

```

\documentclass{article}

% Load the wanted palette.
\usepackage{palettes-hf/Accent}

% Load the palette API.
\usepackage{palapi}

\usepackage{tikz}

\begin{document}

\textcolor{\palUse{Accent}{8}}{\bfseries Colored text.}

Representation of the first ten palette colors.

\begin{tikzpicture}
    \foreach \i in {1,...,10} {
        \fill[\palUse{Accent}{\i}]
            (1.25*\i - 1, 0) rectangle (1.25*\i, 1);
    }
\end{tikzpicture}

\end{document}

```

b. Creating palettes from scratch

Internally, palettes are defined using an array-of-three-floats syntax. For instance, the following snippet is an excerpt from `palettes-hf/Accent.sty`.

```

\palCreateFromRGB{Accent}{
    {0.4980392157, 0.7882352941, 0.4980392157},
    {0.7450980392, 0.6823529412, 0.831372549},
    ...
}

```

For creating new palettes manually, the following high-level commands are available.

1. `\palCreateFromNames` works with a comma separated list of named colors, while `\palCreateFromRGB` creates a palette by entering it as an array-like variable of arrays of three floats.
2. `\palSize{<name>}` returns the palette size (useful for loops, for example).

The following example demonstrates the `\palCreateFromRGB` and `\palCreateFromNames` commands.

```

\usepackage{palapi}
\usepackage[svgnames]{xcolor}

\palCreateFromRGB{MyRGBPal}{
  {0.0, 0.0, 0.0},
  {0.8, 0.2, 0.0},
}

\palCreateFromNames{MyNameUsePal}{
  YellowGreen,
  green!60!black,
}

```

Note.

All built-in palettes are created using the `\palCreateFromRGB` macro.

A lower-level approach is also available through the following commands.

1. `\palNew{<name>}` initializes a new (empty) palette.
2. `\palAddNames{<name>}{<color-using-names>}` appends a color defined with named colors to the palette.
3. `\palAddRGB{<name>}{<r>, <g>, <b>}` appends an RGB color to the palette, where `<r>`, `<g>`, and `<b>` are decimal values ranging from 0 to 1.

The following example demonstrates the flexibility offered by these low-level commands.

```

\usepackage{palapi}
\usepackage[svgnames]{xcolor}

\palNew{LowLevelPal}
\palAddNames{LowLevelPal}{IndianRed}
\palAddRGB{LowLevelPal}{0.0, 0.0, 0.0}
\palAddNames{LowLevelPal}{green!60!black}
\palAddRGB{LowLevelPal}{0.8, 0.2, 0.0}

```

c. Creating palettes from existing ones

Building new palettes by transforming existing ones can be achieved using the `\palCreateFromPal` command, which has the signature `\palCreateFromPal{<new-name>}[<options>]{<existing-name>}`. The following example shows how to do this (all options are used).

```

\usepackage{palettes-hf/Accent}
\usepackage{palapi}

\palCreateFromPal{AccentTransformed}[
  extract = {1, 3, 6, 9},
  shift   = 1,
  reverse
]{Accent}

```

Tip.

`\palCreateFromPal{<new-name>}{<existing-name>}` builds a copy of an existing palette.

d. Retrieving the internal definition of a palette

The internally stored definition of a palette named `MyPal`, for example, is `\g_palette_MyPal_seq` which is a $\LaTeX$ 3 variable (keep in mind the pattern `\g_palette_<palette-name>_seq`).

Note.

Variables of type `\g_palette_<palette-name>_seq` are not used internally to retrieve the colors themselves; they are only there for technical reasons related to the development process of new palettes via *L^AT_EX*.

4. Lua

Note.

Initially, the *@prism* project was created to provide ready-to-use color palettes for *luadraw*, a package that greatly facilitates the creation of high-quality 2D and 3D plots using *LuaL^AT_EX* and *TikZ*.

a. Basic use

The *Lua* palette variables are named using the prefix `pal`. They are arrays of arrays of three floats (making it straightforward to use a color from a palette). For instance, the following snippet is an excerpt from `palettes-hf/Accent.lua`.

```
palAccent = {
  {0.4980392157, 0.7882352941, 0.4980392157},
  {0.7450980392, 0.6823529412, 0.831372549},
  ...
}
```

b. Creating palettes from existing ones

The `palCreateFromPal` function provides options to build new palettes by transforming existing ones. The following example shows how to do this (all options are used).

```
palAccentTransformed = palCreateFromPal(
  palAccent,
  {
    extract = {2, 5, 8, 9},
    shift   = 1,
    reverse = true
  }
)
```

VI. Contribute via Git

Caution.

Never use the `main` branch, which is for freezing the latest stable versions of each projects in the mono repository <https://github.com/projetmbc/for-writing>.

1. Complete the translations

Important.

Although we're going to explain how to translate the documentation, it doesn't seem relevant to do so, as English should suffice these days.

The translations are roughly organized as in figure 1 where just the important folders for the translations have been “opened”.¹⁰ A little further down, the section **VI-1-e** explains how to add new translations.

a. The `en` folder

This folder, managed by the author of *@prism*, contains files easy to translate even if you're not a coder.

¹⁰This was the organization on October 26, 2025.

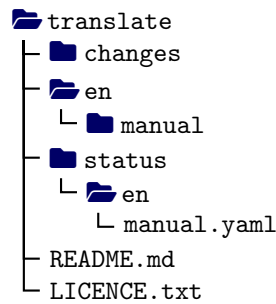


Figure 1: Simplified view of the translation folder

b. The changes folder

This folder is a communication tool where important changes are indicated without dwelling on minor modifications specific to one or more translations.

c. The status folder

This folder is used to keep track of translations from the project's point of view. Everything is done via well-commented YAML files, readable by a non-coder.

d. The README.md and LICENCE.txt files

The LICENCE.txt file is aptly named, while the README.md file takes up in English the important points of what is said in this section about new translations.

e. New translations

Note.

The folder `manual` is reserved for documentation. It contains TEX files that can be compiled directly for real-time validation of translations.

Warning.

Only start from the `en` folder, as it's the responsibility of the `@prism` author.

*Let's say you want to add support for Italian.*¹¹ To do this, you must use `Git` as follows.

1. Via <https://github.com/projetmbc/for-writing/tree/aprism/@prism>, recover the entire project folder. Do not use the `main` branch, which is used to freeze the latest stable versions of all the projects in the mono repository <https://github.com/projetmbc/for-writing>.
2. In the `@prism/contrib/translate` folder, create an `it` copy of the `en` folder, where `it` is the short name of the language documented in [the page “IETF language tag”](#) from Wikipedia.
3. Once the translation is complete in the `it` folder, share it via <https://github.com/projetmbc/for-writing/tree/aprism/@prism> using a classic `git push`.

2. Improving the source code

Participation as a coder is made via the repository <https://github.com/projetmbc/for-writing/tree/aprism/@prism> corresponding to the `@prism` development branch. Here is what you can do, details can be found in the file <https://github.com/projetmbc/for-writing/blob/aprism/@prism/contrib/products/README.md>.

1. Create new palettes within an existing implementation. No coding skills required.
2. Propose a new implementation in your favorite programming language.
3. Combine both approaches.

¹¹As mentioned above, there is no real need for the `doc` folder.

VII. History

2.1.0
2026-03-14

🔧 Fix.

- Online demo: an auto-update script for the `HTML`, `CSS` and `JavaScript` codes was missing.

🔨 Break.

- Palette purification.
 - Deprecated sublist exclusion due to the high overhead of manual reconstruction: return of the palettes `BlueDarkOrange12`, `BlueOrange8`, and `BlueOrange10`.
 - No more visual identical palettes (semi-automated process): `Cubehelix` (`Matplotlib`) was removed, as it is visually identical to `Classic` (`Cubehelix`).

🔄 Update.

- Manual.
 - The `Broc`, `GistRainbow`, and `Tofino` palettes have been removed from the cyclic category.
 - The `BlueDarkOrange12`, `BlueOrange8`, and `BlueOrange10` palettes have been added to the sequential category.
 - Added parameters allowing for the reconstruction of palettes omitted due to being shifted variants of existing ones.
 - Added informative appendix describing subpalettes.
-

2.0.0
2026-02-28

🔨 Break.

- Here are the breaking changes in the API.
 - Palette names use 'semantic' camelcase. For example, we use `PinkGrey` and `BWR` (Blue White Red) instead of `Pinkgrey` and `Bwr`, but we keep `RdBu` (RedBlue).
 - `Lua` product: function `getPal` is now `palCreateFromPal`. It expects a palette variable alongside options.
 - Contribution workflow has been fully changed (see below).
 - No more same size for all the palettes (see below).
 - No sublist or shifted palette.
 - The categories have been fully changed (see below).
- Manual.
 - Ignored and renamed palettes have been placed in dedicated appendices.
 - Remove the cluster appendix.

💎 New.

- Here are the new API features.
 - 95 new palettes added.
 - Multiple distribution formats are now available to meet various technical requirements.
 - * High-fidelity (unmodified) or normalized (size ≤ 40).
 - * One file per palette (no more all palettes in a single file).
 - Categories: the size palette is no longer used since the new API provides named categories. For example, a 3-color smooth palette can be now sequential, qualitative and/or semantic.
 - Contribution workflow simplified significantly.
 - * Data extraction and code generation are now streamlined by instantiating the `PaletteTransformer` class, alongside palette and API builder functions.
 - * Standardized showcases now feature simplified graphics.

🔄 Update.

- Documentations explain the new API. Here are the changes made.
 - Contribution `README.md` files updated.
 - Manual.
 - * Added reference to `CarbonPlan`.
 - * Added new category sections with a small description of their meaning.
- The projects listed below serve as a foundation.
 - `Asymptote`
 - `CarbonPlan` (new)
 - `CARTOColors`
 - `CMasher` (new)
 - `Colorbrewer`
 - `Colormaps` (new, just for category automation)
 - `Matplotlib`
 - `NCAR NCL color tables` (new, partially supported for now)
 - `Palettable`
 - `Scientific Coulour Maps`

1.3.0
2026-01-01

🔧 Fix.

- **Lua** product: the `use-std.tex` and `use-dark.tex` codes have been fixed.

🔧 Break.

- **luadraw** product becomes **Lua** product.
- **Lua** product: the dict-like variable `palNames` has been removed. Therefore, the responsibility of adding this specific variable will lie with **luadraw**.

💎 New.

- New **CSS** and **L^AT_EX** products have been coded.
- The **Palettable** palettes have been added.
- Three semantic palettes **TdocBW**, **TdocCol** and **TdocDark** have been created to be used by the **tutodoc** class (**L^AT_EX** creation process used).
- Added an **online demo of @prism** (the **CSS** product showcase is used).

🔄 Update.

- Manual.
 - Added lists of renamed and removed palettes.
 - Added two appendixes, one with all palettes grouped by user-friendly categories, and another showing similar palettes.
 - Added explanations about the utility of palettes for non-designers.
 - Added explanations about the minimal common optional sets of extra features implemented.
 - Better palette references.
 - Better showcases.
 - Several category **PDF** files are proposed (one **PDF** per category and theme).
 - 2D chart and Delaunay triangulation added.
-

1.2.1
2025-11-09

🔧 Fix.

- Equal palettes: the floating point equality uses now a correct tolerance.

🔧 Break.

- Palettes: the extra **Greys** has been removed (it is equal to **Grays**).

💎 New.

- Similar palettes: two PDF files show similar palettes in standard and black modes (semi-automated process used).

🔄 Update.

- **luadraw** product: the associative array `palNames` has been added for compatibility reasons with the **luadraw** package.
 - **BlindFish** palette: the last color variation has been made smoother (**luadraw** process used).
-

1.2.0
2025-10-29

🔧 Break.

- Palettes: all final palettes now consist of 10 colors.
- **luadraw** products: the `getPal` dictionary array has been converted into a function accepting string palette names (with or without `pal` prefix). See below.

💎 New.

- Palettes.
 - Added **Lemon** and **ShiftRainbow** palettes (**luadraw** creation process used).
 - Added 37 palettes from the **Scientific Coulour Maps** project.
 - **luadraw** product: accessing a palette and creating new ones can be made using the `getPal` function which has an optional argument `options` (dict-like array) with the following keys and their values.
 - **extract**: a list of non-zero integers used to extract specific colors from the palette (the order is preserved).
 - **reverse**: a boolean value indicating whether to reverse the palette color order (**false** by default).
 - **shift**: an integer value for applying a circular color shift to the palette.
 - Documentations
 - Added English PDF manual.
 - Showcase: two PDF files demonstrate the use of each palette (white and dark modes).
-

Break.

- Duplicate palettes and those that are reverse of others are ignored (strict equalities only).

New.

- New palettes added: `BurningGrass`, `GeoRainbow` and `PastelRainbow` (`luadraw` creation process used).
- The `luadraw` palette product has a new dictionary like variable `getPal` to access a palette using its name (as a string variable).

Update.

- Palette contributions: in the mandatory `extend.py` file, the `build_code` function must work with the dictionary of all the palettes, and manage a credit to the `@prism` project.

First public version of the project.

Appendix 1 – All palette names

Important.

The palette names in this appendix are standard, but some @prism implementations add a specific prefix.

A

Accent
Acton
AfmHot
AgGrnYl
AgSunset
Algae
Alphabet
Amber
Amethyst
Amp
Antique
Apple
Aquatic1
Aquatic2
Aquatic3
Arctic
ArmyRose
Autumn

B

Balance
Bam
Bamako
BamO
Batlow
BatlowK
BatlowW
Berlin
Bilbao
Binary
Blackbody
BlindFish
BlueDarkOrange12
BlueDarkOrange18
BlueDarkRed12
BlueDarkRed18
BlueGray
BlueGreen
BlueGrey
BlueOrange8
BlueOrange10
BlueOrange12
BlueOrangeRed
BlueRed

BlueRedPLY

Blues
Blues7
Blues10
BluesCP
BluGrn
BluYl
Bold
Bone
BrBG
BRG
Broc
BrocO
BrownBlue10
BrownBlue12
BrwnYl
Bubblegum
Buda
BuGn
Bukavu
BuPu
Burg
BurgYl
BurningGrass
BWR

C

Cavalcanti
Chevalier
Chroma
Cividis
Classic
CMRmap
ColorBlind
Cool
CoolCP
CoolWarm
Copper
CopperCM
Cork
CorkO
Cosmic
Cube1
Cubehelix1
Cubehelix2

Cubehelix3

CubeYF
Curl

D

Darjeeling1
Darjeeling2
Darjeeling3
Darjeeling4
Dark2
Dark24
DarkMint
Davos
Deep
Delta
Dense
Devon
Dusk

E

Earth
EarthCP
Eclipse
Electric
Ember
Emerald
Emergency
Emrld

F

Fall
FallCM
FantasticFox1
FantasticFox2
Fes
Fire
Flag
Flamingo
Freeze
Fusion

G

G10
GasFlame
Gem
GeoRainbow
Geyser
GhostLight
GistEarth

GistGray
 GistHeat
 GistNcar
 GistRainbow
 GistStern
 Glasgow
 GnBu
 Gnuplot
 Gnuplot2
 Gothic
 GrandBudapest1
 GrandBudapest2
 GrandBudapest3
 GrandBudapest4
 GrandBudapest5
 Gray
 GrayC
 GreenGrey
 GreenMagenta
 GreenOrange
 Greens
 GreensCP
 Greys
 GreysCP
 Guppy

H

Haline
 Hawaii
 Heart
 Holly
 Horizon
 Hot
 HotPLY
 HSV

I

Ice
 IceBurn
 Imola
 Inferno
 Infinity
 IsleOfDogs1
 IsleOfDogs2
 IsleOfDogs3

J

Jet
 JetPLY
 JimSpecial
 Jungle

L

Lajolla
 Lapaz
 Lavender
 Lemon
 Light24
 Lilac
 LinearL
 Lipari
 Lisbon

M

Magenta
 Magma
 Managua
 Margot1
 Margot2
 Margot3
 Matter
 Mendl
 MidGray
 Mint
 Moonrise1
 Moonrise2
 Moonrise3
 Moonrise4
 Moonrise5
 Moonrise6
 Moonrise7

N

Navia
 NaviaW
 Neon
 NipySpectral
 Nuclear
 Nuuk

O

Ocean
 OceanCM
 OkabeIto
 Oleron
 OrangeBlue
 OrangeGrey
 Oranges

OrangesCP
 OrRd
 OrYel
 Oslo
 Oxy

P

Paired
 Pastel
 Pastel1
 Pastel2
 PastelRainbow
 Peach
 Pepper
 PerceptualRainbow
 Petroff6
 Petroff8
 Petroff10
 Phase
 Picnic
 Pink
 PinkGreen
 PinkGrey
 Pinks
 PinkYl
 PiYG
 Plasma
 Plotly
 Portland
 PrGn
 Pride
 Prinsenvlag
 Prism
 PrismCC
 PuBu
 PuBuGn
 PuOr
 PuRd
 Purp
 Purple
 PurpleGray
 PurpleGrey
 Purples
 PurplesCP
 PurpOr

R

Rainbow
 RainbowCP
 RainbowPLY
 Rainforest

RdBu
RdGy
RdPu
RdYlBu
RdYlGn
Red
RedGrey
RedOr
Reds
RedsCP
RedShift
RedTeal
RedYellowBlue
Roma
RomaO
Royal1
Royal2
Royal3

S

Safe
Sapphire
Seasons
Seaweed
Seismic
Sepia
Set1
Set2
Set3
ShiftRainbow
SineBow
Solar
Spectral
Speed
Spring
Summer
SunBurst
Sunset
SunsetDark
Swamp

T

T10
Tab10
Tab20
Tab20b
Tab20c
TableauLight
TableauMedium
TdocBW
TdocCol

TdocDark
Teal
TealGrey
TealGrn
TealRose
Teals
Tempo
Temps
Terrain
Thermal
Tofino
Tokyo
Torch
Toxic
TrafficLight
Tree
Tropic
Tropical
Turbid
Turbo
Turku
Twilight

V

Vanimo
Vik
VikO
Viola
Viridis
Vivid
Voltage

W

Warm
Water
Waterlily
Watermelon
Wildfire
Wind
Winter
Wistia

Y

YellowGrey
YellowPurple
Yellows
YlGn
YlGnBu
YlOrBr
YlOrRd
YlOrRdCBW

Z

Zissou

Appendix 2 – Renamed palettes

Important.

The palette names in this appendix are standard, but some `@prism` implementations add a specific prefix.

To prevent name conflicts, specific suffixes can be applied. Here is the full list of the suffixes used.

Table 1: Suffixes used

CarbonPlan: CP	Colorbrewer: CBW
CARTOColors: CC	Plotly: PLY
CMasher: CM	

Below is the list of all renamed palettes. We use $\Rightarrow$ to denote these changes, with the new `@prism` names appearing on the right.

Table 2: Renamed palettes

CarbonPlan	Bluegrey	$\Rightarrow$	BlueGrey
	Blues	$\Rightarrow$	BluesCP
	Cool	$\Rightarrow$	CoolCP
	Earth	$\Rightarrow$	EarthCP
	Greengrey	$\Rightarrow$	GreenGrey
	Greens	$\Rightarrow$	GreensCP
	Greys	$\Rightarrow$	GreysCP
	Orangeblue	$\Rightarrow$	OrangeBlue
	Orangegrey	$\Rightarrow$	OrangeGrey
	Oranges	$\Rightarrow$	OrangesCP
	Pinkgreen	$\Rightarrow$	PinkGreen
	Pinkgrey	$\Rightarrow$	PinkGrey
	Purplegrey	$\Rightarrow$	PurpleGrey
	Purples	$\Rightarrow$	PurplesCP
	Rainbow	$\Rightarrow$	RainbowCP
	Redgrey	$\Rightarrow$	RedGrey
	Reds	$\Rightarrow$	RedsCP
	Redteal	$\Rightarrow$	RedTeal
	Sinebow	$\Rightarrow$	SineBow
	Tealgrey	$\Rightarrow$	TealGrey
	Yellowgrey	$\Rightarrow$	YellowGrey
	Yellowpurple	$\Rightarrow$	YellowPurple
CARTOColors	Prism	$\Rightarrow$	PrismCC
CMasher	Copper	$\Rightarrow$	CopperCM
	Fall	$\Rightarrow$	FallCM
	Ghostlight	$\Rightarrow$	GhostLight
	Iceburn	$\Rightarrow$	IceBurn
	Ocean	$\Rightarrow$	OceanCM
	Redshift	$\Rightarrow$	RedShift

Continued on next page

Table 2: Renamed palettes (Continued)

	Sunburst	⇒	SunBurst
Colorbrewer	PRGn	⇒	PrGn
	YlOrRd	⇒	YlOrRdCBW
Matplotlib	Afmhot	⇒	AfmHot
	Brg	⇒	BRG
	Bwr	⇒	BWR
	Coolwarm	⇒	CoolWarm
	Hsv	⇒	HSV
Plotly	Bluered	⇒	BlueRedPLY
	Hot	⇒	HotPLY
	Jet	⇒	JetPLY
	Rainbow	⇒	RainbowPLY
Tableau	Gray	⇒	MidGray

Appendix 3 – Ignored palettes

Important.

The palette names in this appendix are standard, but some `@prism` implementations add a specific prefix.

Excluded by design

The following palettes have been intentionally excluded. Please refer to the explanation below.

Table 3: Excluded palettes

CMasher	Neutral	Too big (size > 512).
	Savanna	Looks completely black.

Unkept palettes

The following types of palettes are unkept.

- **Exact duplicates:** identical color arrays.
- **Visual identical:** those indistinguishable to the naked eye.¹²
- **Transformations:** reversals, shifts, or a combination of both.

We identify relations between palettes using the following notations.

Duplicates



Visually identical



Reversal



Reversal and shifted



We list the unkept palettes below, omitting duplicates that share both name and colors. The rightmost palettes are those retained in `@prism`. For instance, the table shows that `Matplotlib`'s `Gray` is the reversal of `@prism`'s `Binary`.

Table 4: Unkept palettes

Matplotlib	Cubehelix	≅	Classic
	TwilightShifted	⇌	Twilight
	Gray	⇌	Binary
	GistYarg	⇌	GistGray
Plotly	D3	=	Tab10
Tableau	Tableau	=	Tab20

Reconstructing unkept shifted palettes

Let's detail how to retrieve only those excluded palettes from `@prism` that required a shift.

Table 5: Unkept palettes

Matplotlib	TwilightShifted	pal_to_use = Twilight
		reverse = True
		shift = 256

¹²In these cases, the original palette is retained. For instance, as `Matplotlib`'s `Cubehelix` is a specific implementation of the original `Cubehelix`'s `Classic`, we keep the latter.

Appendix 4 – Subpalettes

Important.

The palette names in this appendix are standard, but some @prism implementations add a specific prefix.

For informational purposes, this section lists palettes included within others and describes the process for reconstructing these subpalettes.

Table 6: Unkept palettes

Binary	pal_to_use = Flag indices = 85, 93
BlueRedPLY	pal_to_use = BRG indices = 0, 1
BWR	pal_to_use = Flag indices = 73, 85, 97
Cool	pal_to_use = ShiftRainbow indices = 0, 2
Speed	pal_to_use = Delta range = from 256 to 511
Tab10	pal_to_use = Tab20 range = from 0 to 18 with step 2
TableauLight	pal_to_use = Tab20 range = from 1 to 19 with step 2

Appendix 5 – Palettes by category

Important.

The palette names in this appendix are standard, but some @prism implementations add a specific prefix.

Note (Qualitative and semantic palettes).

The threshold of 40 colors has been established for qualitative and semantic palettes. The following lines outline the rationale for this limit.

- In qualitative color palettes, each color represents a single data point or a group of data. Experience shows that about ten colors are more than enough.*
- Semantic palettes associate a color with a specific context, such as warning levels (see the frames in this document). Choosing 40 colors allows for 10 different contexts, each with 4 sub-categories. Here are some examples of sub-categories.*

10-19 Initial / Idle: default states such as “To Do”, “Clickable”, or “To be analyzed”.

20-29 Active / Transition: states in flux, including “In Progress” or UI “Hover” effects.

30-39 Success / Final: terminal positive states like “Done”, “Clicked”, or “Validated”.

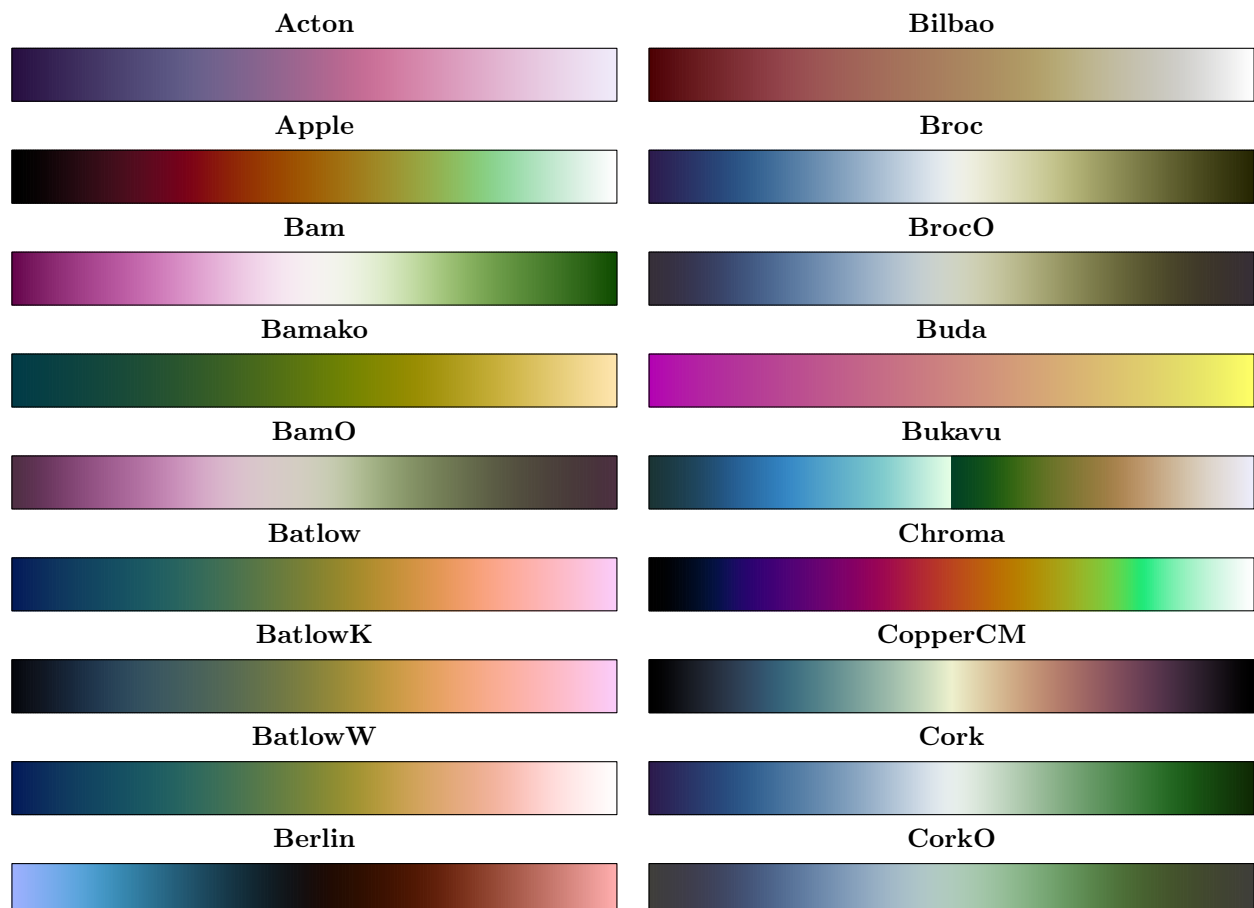
40-49 Alert / Rejection: error handling and negative outcomes, e.g., “Error”, “Inactive”, or “Rejected”.

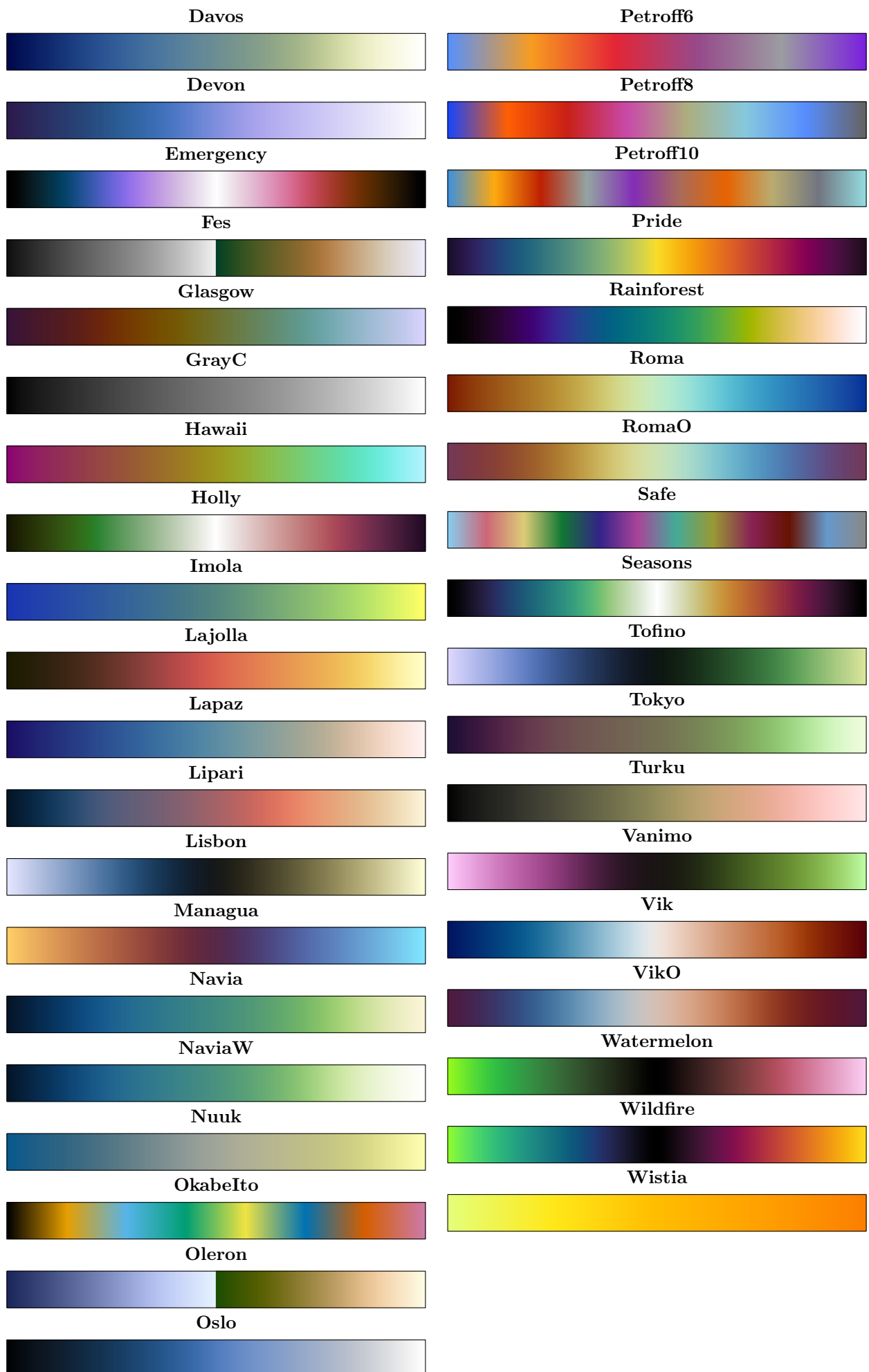
Important.

Categories were generated semi-automatically, followed by manual selection to obtain relevant choices.

Colorblind palettes

Colorblind palettes provide high-contrast hues specifically optimized for users with color vision deficiency.

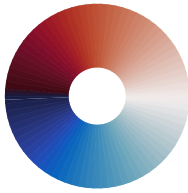




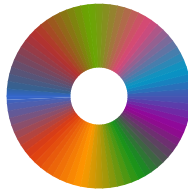
Cyclic palettes

Cyclic palettes create seamless loops for periodic data, such as angles, seasons, or 24-hour cycles.

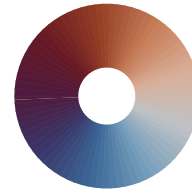
Balance



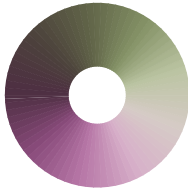
G10



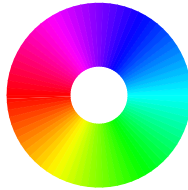
VikO



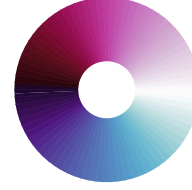
BamO



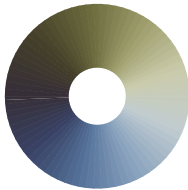
HSV



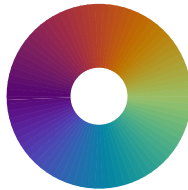
Viola



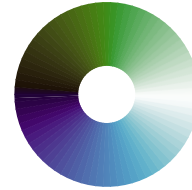
BrocO



Infinity



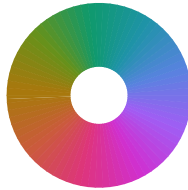
Waterlily



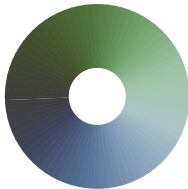
CopperCM



Phase



CorkO



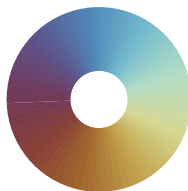
Pride



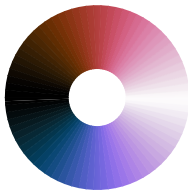
Delta



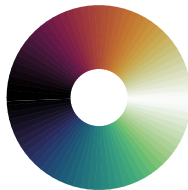
RomaO



Emergency



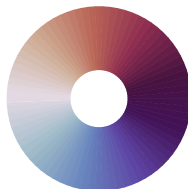
Seasons



Fusion

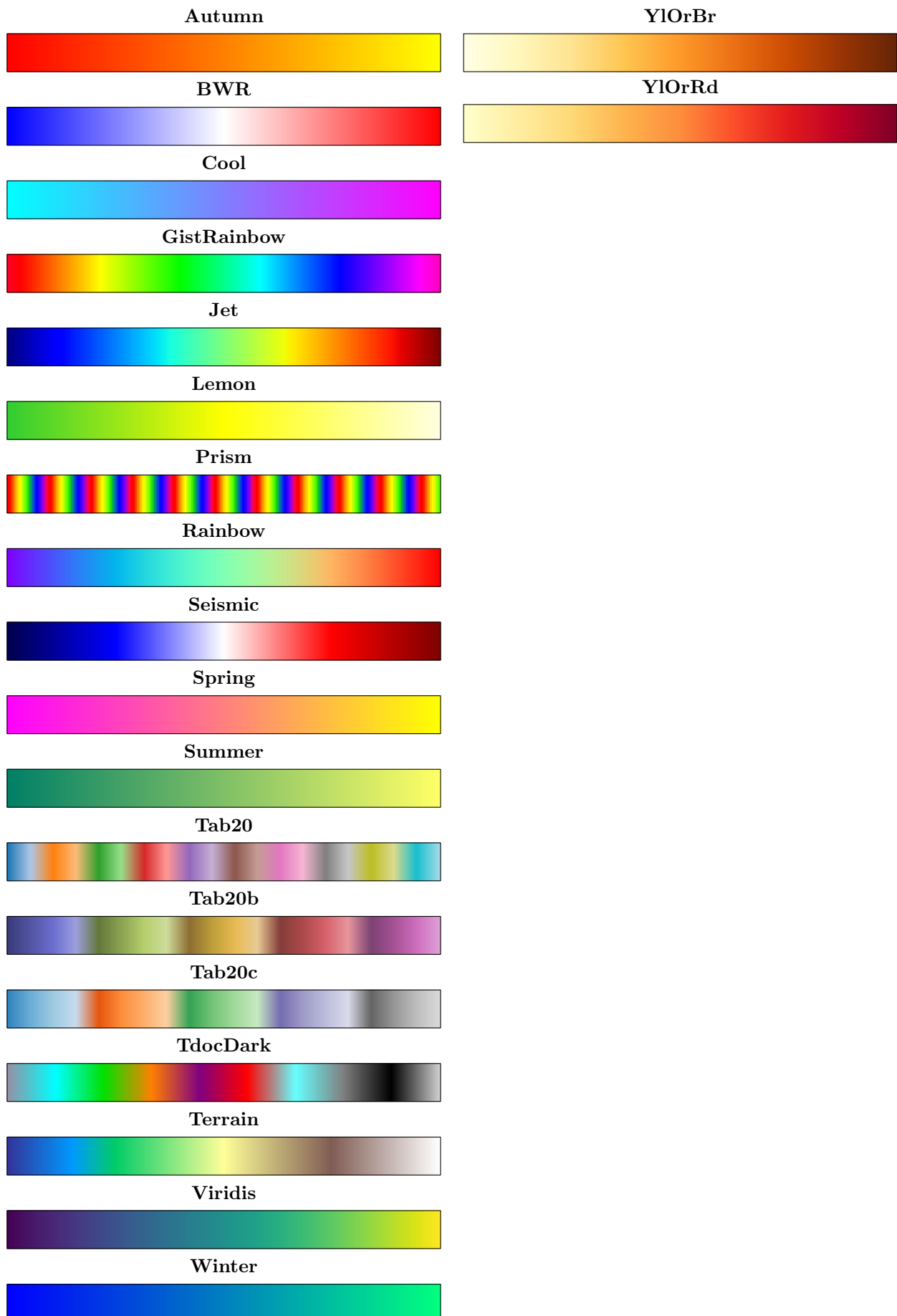


Twilight



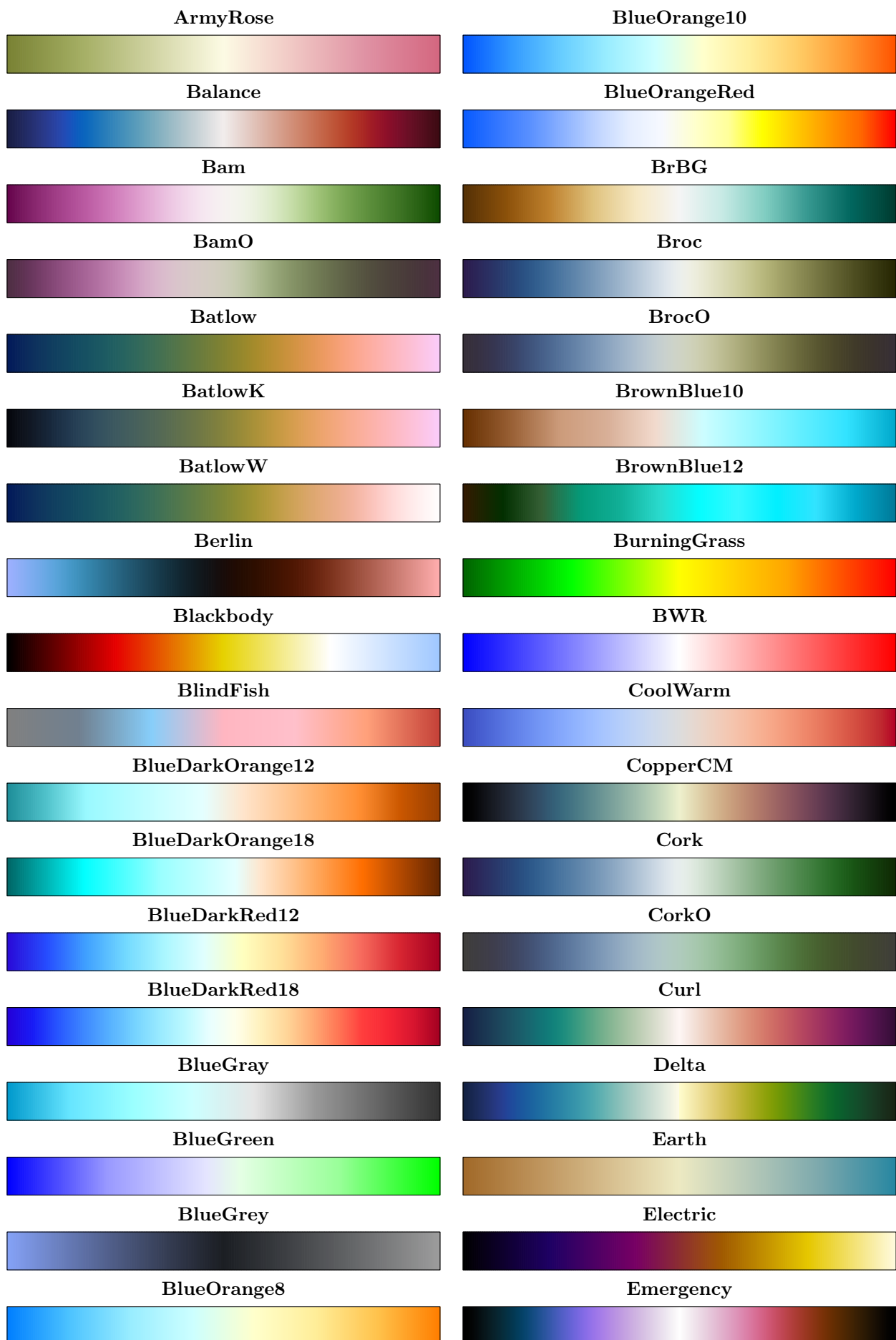
Dark palettes

Dark palettes offer high-contrast schemes specifically optimized for terminal interfaces and dark-mode backgrounds.



Divergent palettes

Divergent palettes highlight deviations from one midpoint toward contrasting extremes.





TealRose



Temps



Tofino



Tropic



Twilight



Vanimo



Vik



VikO



Viola



Waterlily



Watermelon



Wildfire



YellowGrey

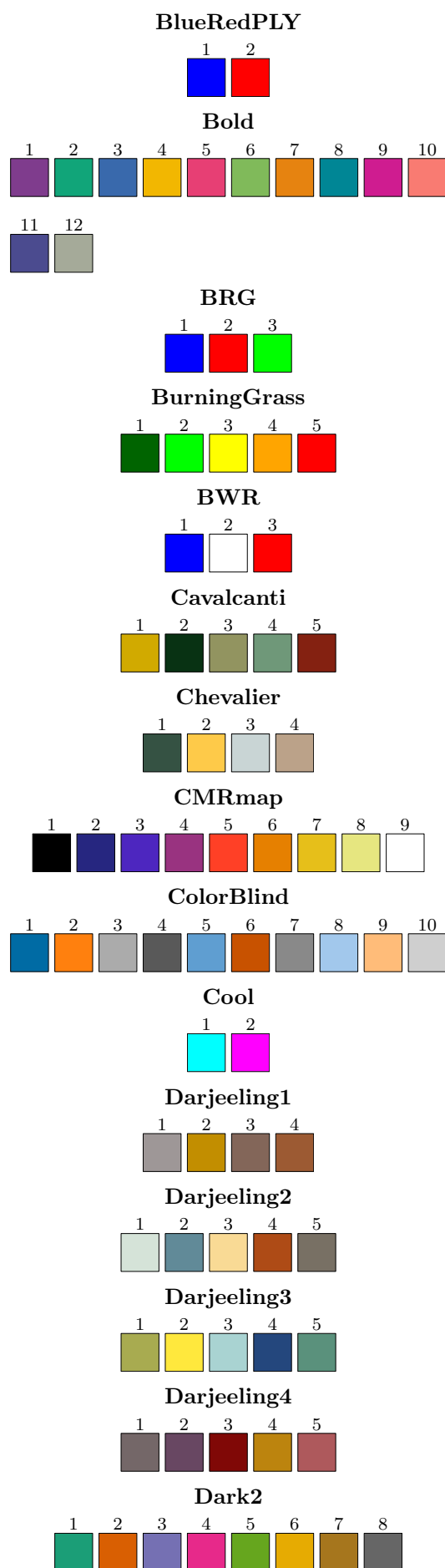
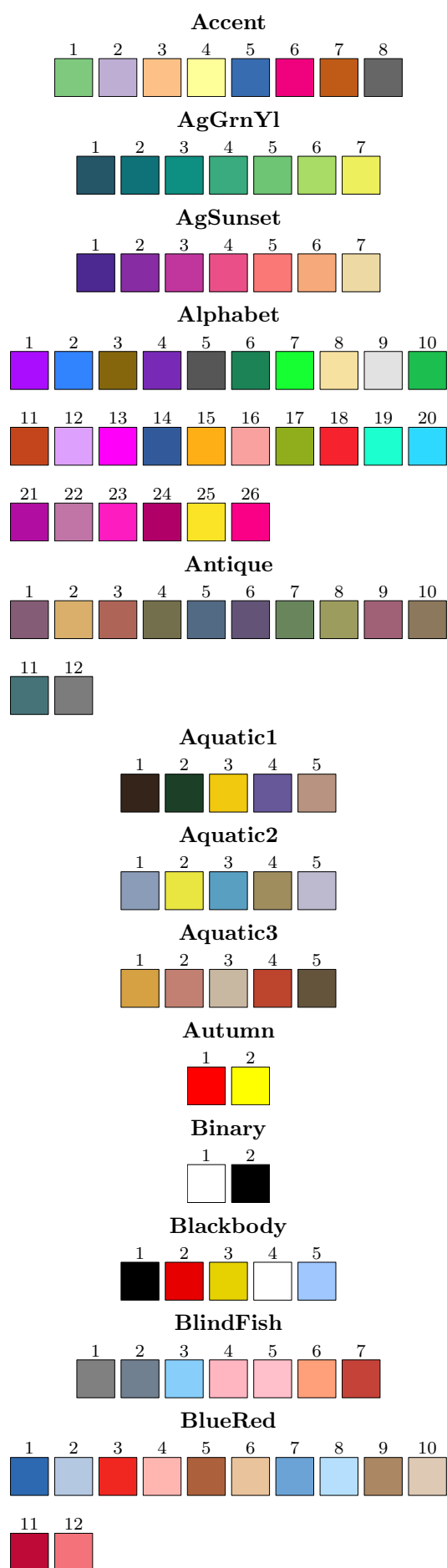


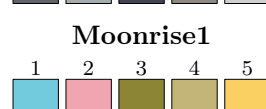
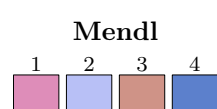
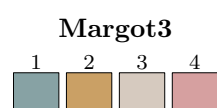
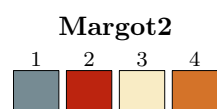
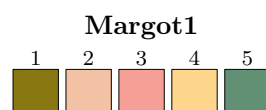
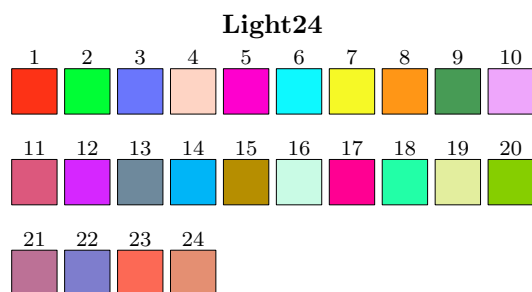
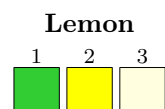
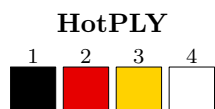
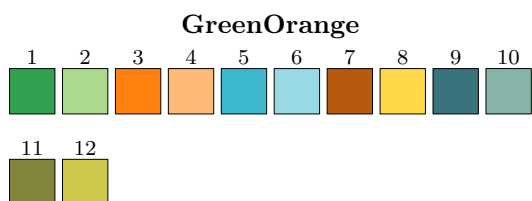
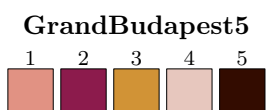
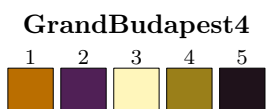
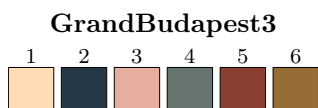
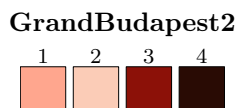
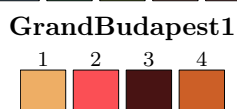
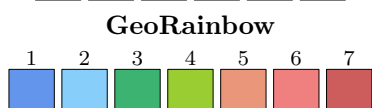
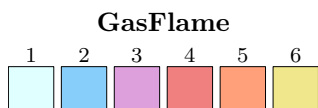
YellowPurple

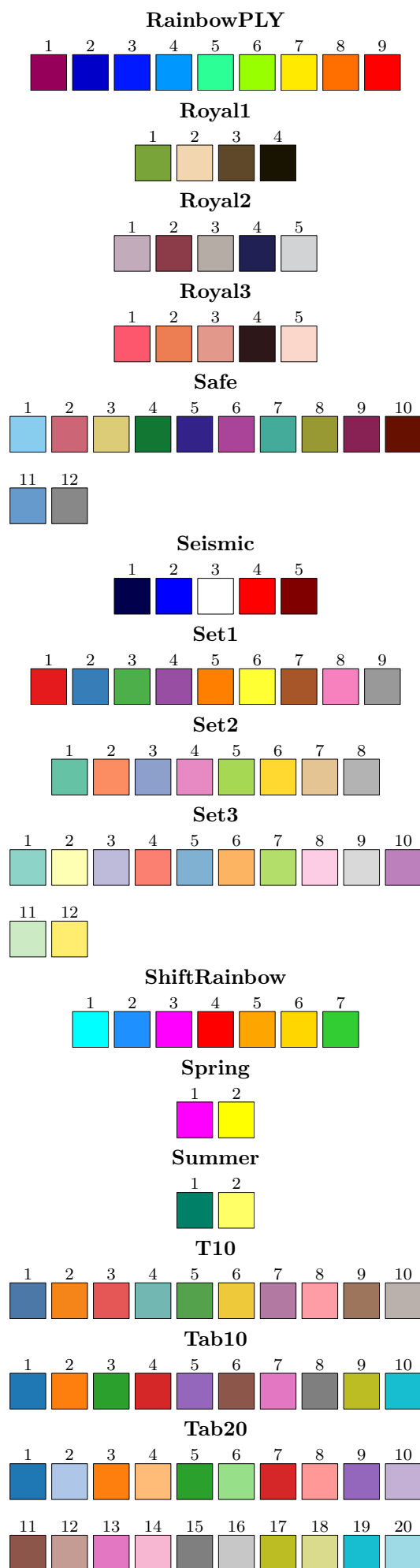
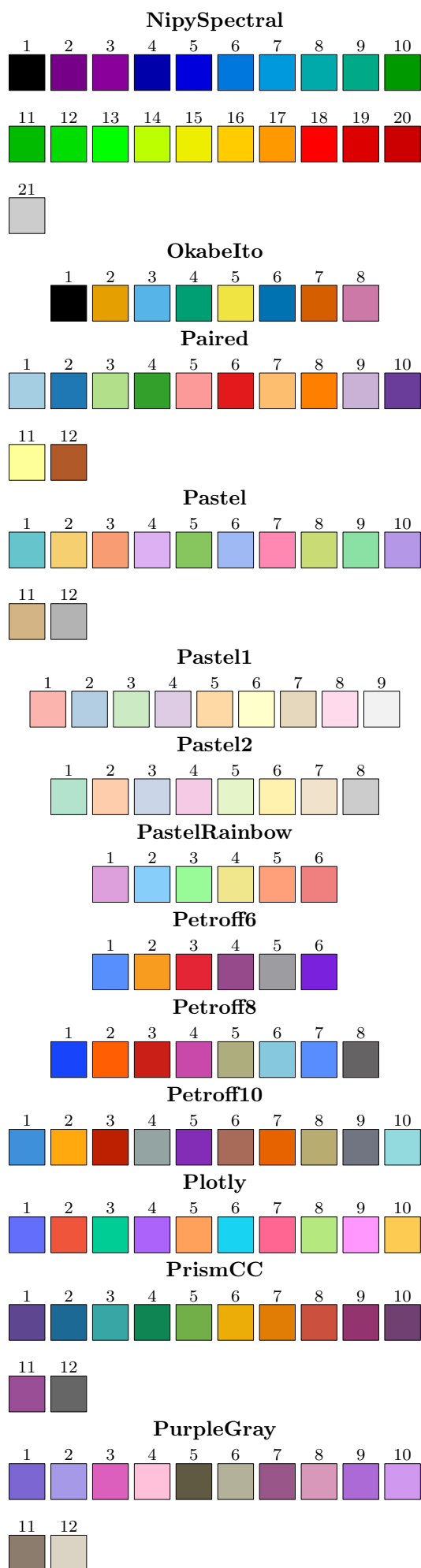


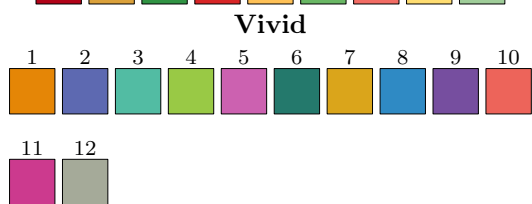
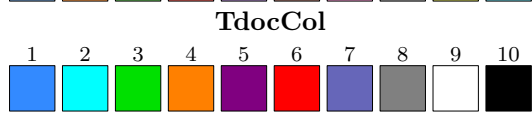
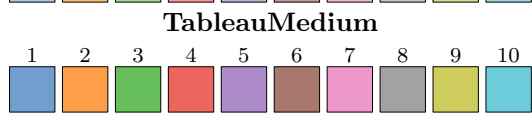
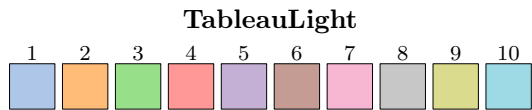
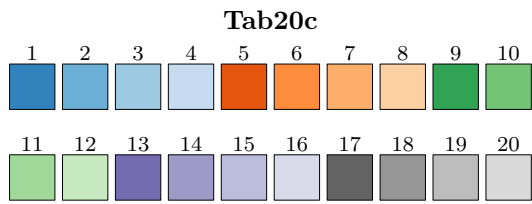
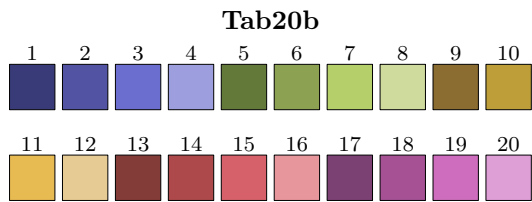
Qualitative palettes

Qualitative palettes use distinct hues to separate categorical data without implying any order or hierarchy.

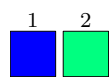








Winter

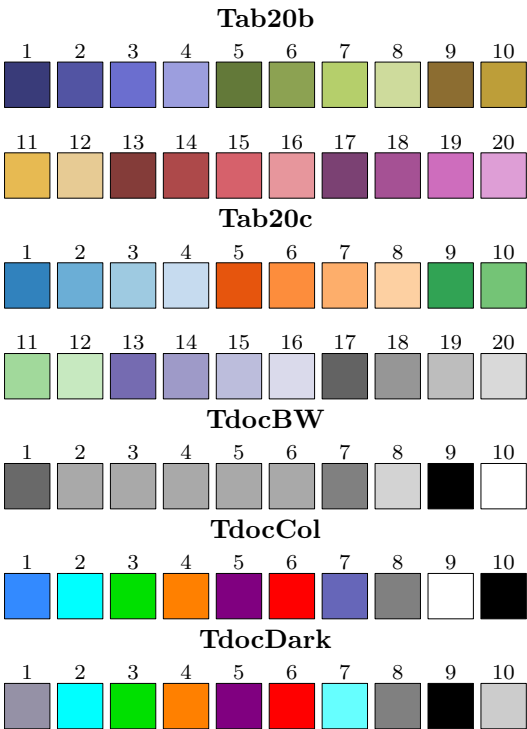


Zissou



Semantic palettes

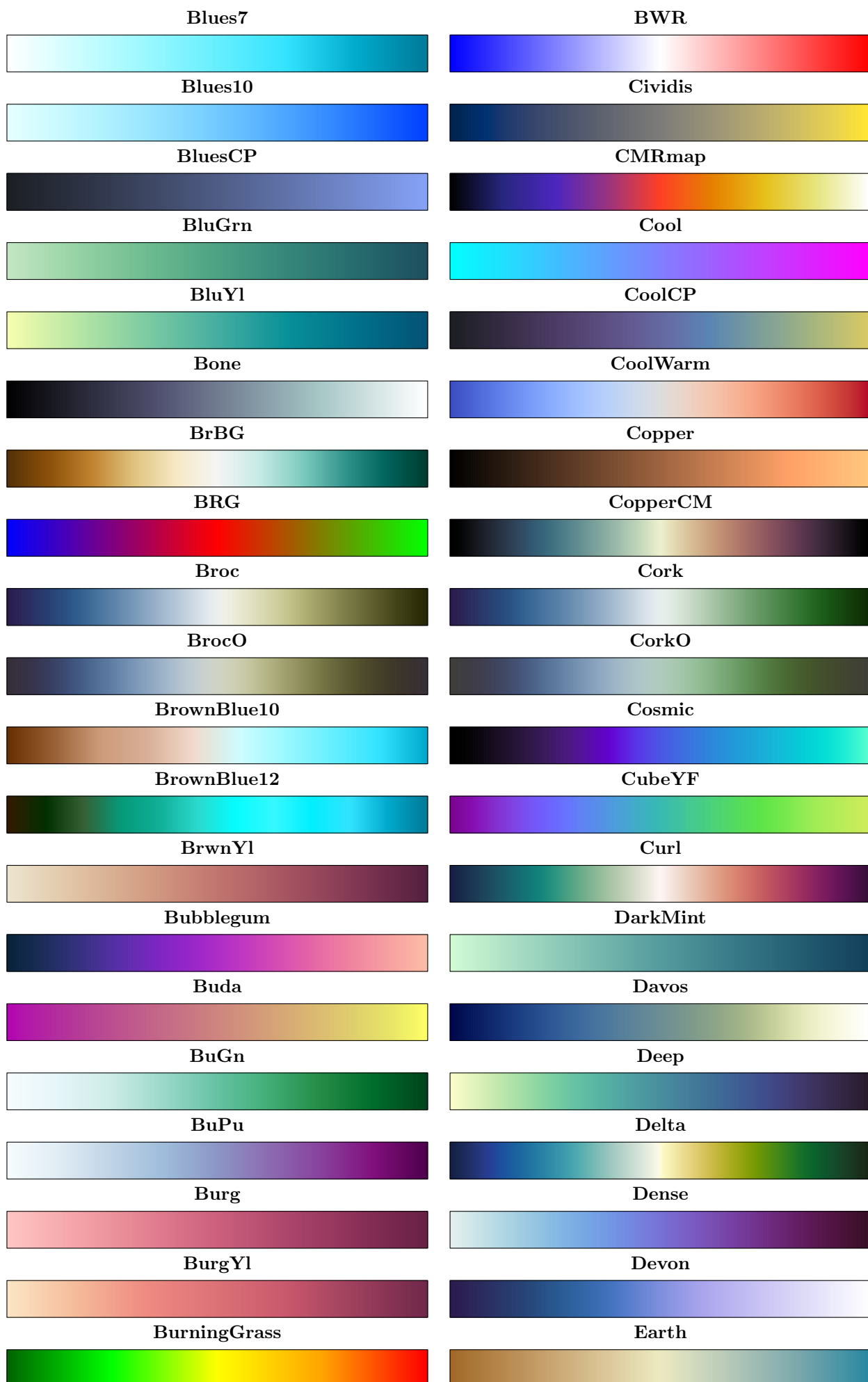
Semantic palettes are designed to assign colors to categories — such as UI states, system status, or pedagogical elements — to convey immediate meaning.

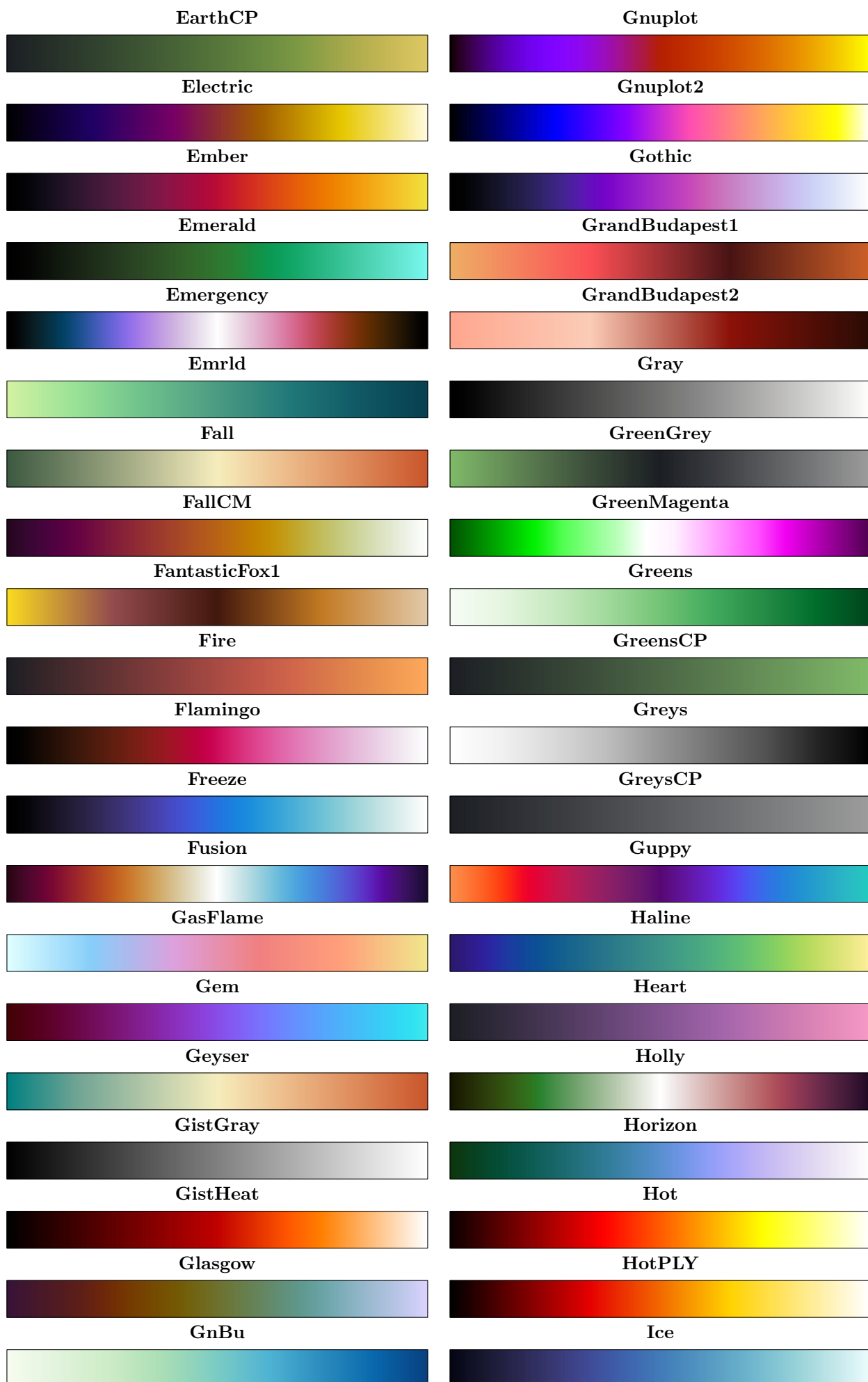


Sequential palettes

Sequential palettes represent ordered data using a single or multi-hue gradient with monotonic lightness.













YlOrRdCBW



Stepped palettes

Stepped palettes are designed to highlight distinct shifts rather than smooth progression. They use high-contrast colors that meet at sharp boundary points to emphasize deviation.

